



计算机组成与体系结构实验

实验二报告

学 院	计算机学院
专 业	计算机科学与技术
指导老师	李博扬
姓 名	左逸龙
学 号	1120231863
联系方式	cs.yilong.zuo@bit.edu.cn

二〇二六年六月

目 录

第一章 项目简述	1
第二章 设计目的	1
第三章 设计环境	1
第四章 设计原理及内容	1
4.1 指令子集	1
4.2 CPU 整体结构	2
4.3 模块划分	3
4.4 控制信号	3
第五章 设计与实现	4
5.1 排序程序	4
5.2 机器码	6
5.3 关键实现	6
第六章 测试	7
6.1 RARS 运行结果	7
6.2 CPU 仿真方法	7
6.3 结果对比	8
第七章 问题及解决方法	9
第八章 心得体会及总结	9

第一章 项目简述

本实验实现了一个面向 RV32I 子集的单周期 CPU。处理器采用取指、译码、执行、访存、写回在一个时钟周期内完成的结构，指令存储器和数据存储器分开实现，寄存器堆包含 32 个 32 位通用寄存器，其中 x0 固定为 0。

本 CPU 支持排序程序中用到的 xor、addi、ori、jal、lw、sw、bge、blt 等指令，同时实现 add、sub、beq，并补充 lui、bne 方便后续扩展。测试程序对 5 个整数 4, 2, 5, 1, 3 做冒泡排序，RARS 与单周期 CPU 的仿真结果均为 1, 2, 3, 4, 5。

第二章 设计目的

本实验主要练习从 RISC-V 指令编码到数据通路控制信号的完整设计过程。相比实验一只实现 ALU，本实验需要把 PC、指令存储器、立即数扩展、寄存器堆、ALU、数据存储器和分支跳转逻辑连成完整处理器，因此要同时检查运算模块、操作数选择、写回数据和下一条 PC。

排序程序中既有访存指令，也有条件分支和无条件跳转，因此用它测试 CPU 可以同时覆盖寄存器读写、地址计算、内存交换和循环控制，比只测试单条算术指令更容易暴露数据通路当中的连接错误。

第三章 设计环境

表 3-1 实验环境

项目	内容
操作系统	Microsoft Windows 10.0.19045 x64
硬件描述语言	Verilog HDL
EDA 工具	Vivado 2019.2
RISC-V 仿真器	RARS 1.6

第四章 设计原理及内容

4.1 指令子集

实验任务书要求至少支持 lw、sw、add、sub、ori、beq 等 RV32I 整数指令。为了运行 5 个数排序程序，本实验在此基础上加入 xor、addi、jal、blt、bge，并保留 lui 和 bne 作为同类指令扩展。指令功能见表 4-1。

表 4-1 指令功能与数目

序号	操作码	助记符	功能	描述
1	011 0011	add	$R[rd] \leftarrow R[rs1] + R[rs2]; PC \leftarrow PC + 4$	寄存器加法
2	011 0011	sub	$R[rd] \leftarrow R[rs1] - R[rs2]; PC \leftarrow PC + 4$	寄存器减法
3	011 0011	xor	$R[rd] \leftarrow R[rs1] \wedge R[rs2]; PC \leftarrow PC + 4$	按位异或
4	001 0011	addi	$R[rd] \leftarrow R[rs1] + imm; PC \leftarrow PC + 4$	立即数加法
5	001 0011	ori	$R[rd] \leftarrow R[rs1] \mid imm; PC \leftarrow PC + 4$	立即数按位或
6	011 0111	lui	$R[rd] \leftarrow imm[31:12] \ll 12; PC \leftarrow PC + 4$	装载高位立即数
7	000 0011	lw	$R[rd] \leftarrow Mem[R[rs1] + imm]; PC \leftarrow PC + 4$	字加载
8	010 0011	sw	$Mem[R[rs1] + imm] \leftarrow R[rs2]; PC \leftarrow PC + 4$	字存储
9	110 0011	beq	if $R[rs1] == R[rs2]$ then $PC \leftarrow PC + imm$ else $PC \leftarrow PC + 4$	相等分支
10	110 0011	bne	if $R[rs1] \neq R[rs2]$ then $PC \leftarrow PC + imm$ else $PC \leftarrow PC + 4$	不等分支
11	110 0011	blt	if $R[rs1] < R[rs2]$ then $PC \leftarrow PC + imm$ else $PC \leftarrow PC + 4$	有符号小于分支
12	110 0011	bge	if $R[rs1] \geq R[rs2]$ then $PC \leftarrow PC + imm$ else $PC \leftarrow PC + 4$	有符号大于等于分支
13	110 1111	jal	$R[rd] \leftarrow PC + 4; PC \leftarrow PC + imm$	无条件跳转并链接
指令总 数目		13 条		

4.2 CPU 整体结构

本 CPU 采用单周期数据通路。PC 给出取指地址，指令存储器读出 32 位指令后，控制器根据 opcode、funct3 和 funct7 产生控制信号。寄存器堆读出 rs1、rs2，立即数生成器按指令类型生成 I/S/B/U/J 型立即数。ALU 完成算术逻辑运算和访存地址计算，数据存储器处理 lw、sw，写回多路选择器在 ALU 结果、数据存储器读数、PC+4 和 U 型立即数之间选择写回值。分支单元根据寄存器比较结果决定下一条 PC。

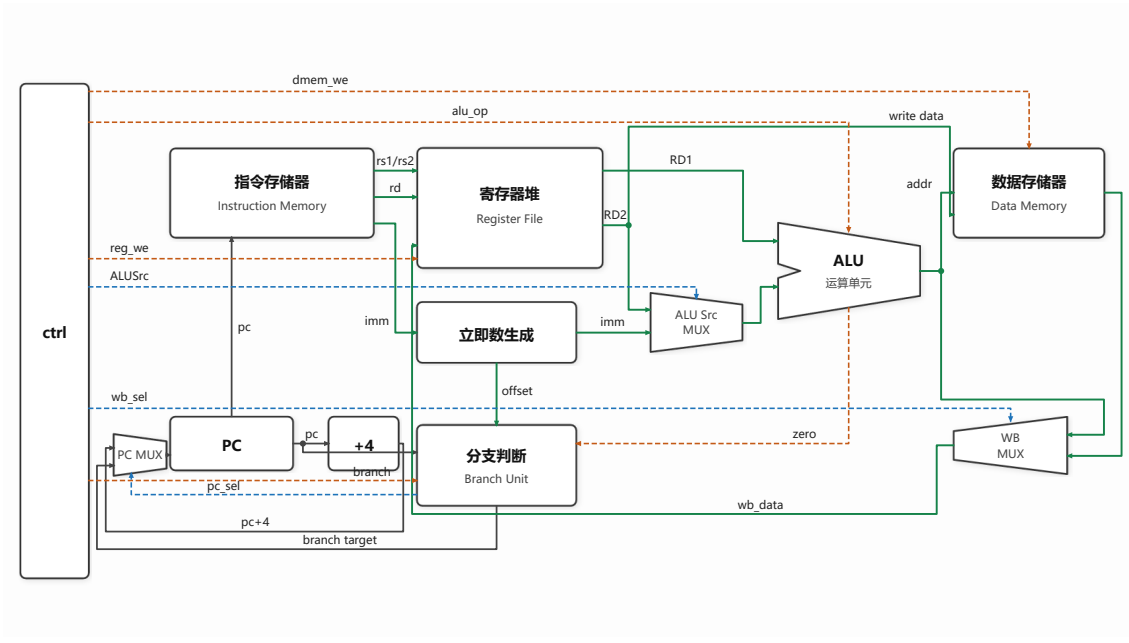


图 4-1 单周期 CPU 数据通路

4.3 模块划分

处理器按功能划分为以下模块。每个模块只处理一种数据通路职责，后面看波形时也能直接按模块定位错误。

表 4-2 CPU 模块划分

模块	功能
ProgramCounter	保存当前 PC，复位时回到 0，每个时钟上升沿写入 next_pc。
InstructionMemory	初始化指令存储器，按 pc[9:2] 做字寻址。
RegisterFile	实现 32 个通用寄存器，组合读、时钟上升沿写，x0 恒为 0。
ImmediateGenerator	按 I/S/B/U/J 型格式生成 32 位符号扩展立即数。
ControlUnit	根据指令编码产生写回、访存、ALU、立即数和分支控制信号。
ALU	完成加、减、或、异或等运算。访存地址也由 ALU 加法得到。
DataMemory	初始化数据存储器，支持组合读和同步写。
BranchUnit	处理 beq、bne、blt、bge 和 jal 的 PC 选择。

4.4 控制信号

控制器采用组合逻辑直接译码。由于本实验指令数量不多，用 `case(opcode)` 再结合 `funct3/funct7` 的写法比较直接，也便于从波形里检查某条指令对应的控制输出。主要控制信号见表 4-3。

表 4-3 CPU 控制信号列表

操作码 op	0110011	0110011	0110011	0010011	0010011	0110111	0000011
信号/指令	add	sub	xor	addi	ori	lui	lw
RegWrite	1	1	1	1	1	1	1
MemWrite	0	0	0	0	0	0	0
ALUSrc	0	0	0	1	1	0	1
WBSel	ALU	ALU	ALU	ALU	ALU	IMM	MEM
ImmType	None	None	None	I	I	U	I
BranchOp	None	None	None	None	None	None	None
ALUOp	ADD	SUB	XOR	ADD	OR	ADD	ADD

操作码 op	0100011	1100011	1100011	1100011	1100011	1101111
信号/指令	sw	beq	bne	blt	bge	jal
RegWrite	0	0	0	0	0	1
MemWrite	1	0	0	0	0	0
ALUSrc	1	0	0	0	0	0
WBSel	/	/	/	/	/	PC4
ImmType	S	B	B	B	B	J
BranchOp	None	BEQ	BNE	BLT	BGE	JAL
ALUOp	ADD	ADD	ADD	ADD	ADD	ADD

组合逻辑译码时，先由 opcode 判断指令大类，再由 funct3/funct7 区分同一类中的具体操作。主要控制信号可按下列关系得到：

```

1 R_type = (opcode == 0110011)
2 I_type = (opcode == 0010011)
3 Load  = (opcode == 0000011)
4 Store  = (opcode == 0100011)
5 Branch = (opcode == 1100011)
6 Lui    = (opcode == 0110111)
7 Jal    = (opcode == 1101111)
8
9 RegWrite = R_type | I_type | Load | Lui | Jal
10 MemWrite = Store
11 ALUSrc   = I_type | Load | Store
12 WBSel    = MEM when Load, PC4 when Jal, IMM when Lui, otherwise ALU
13 ImmType  = I/S/B/U/J according to instruction format
14 BranchOp = funct3 when Branch, JAL when Jal, otherwise None
15 ALUOp    = SUB/XOR/OR according to funct3/funct7, otherwise ADD

```

第五章 设计与实现

5.1 排序程序

测试程序采用冒泡排序。数据初值放在数据存储器前五个字中，程序每轮从后往前比较相邻元素，如果右侧元素小于左侧元素，就用两条 sw 交换。外层指针 t0 逐步右移，内层指针 t1 从最后一个元素向前移动。程序中 blt 和 bge 都按有符号数比较，本实验数据为正数，因此结果与无符号比较一致。

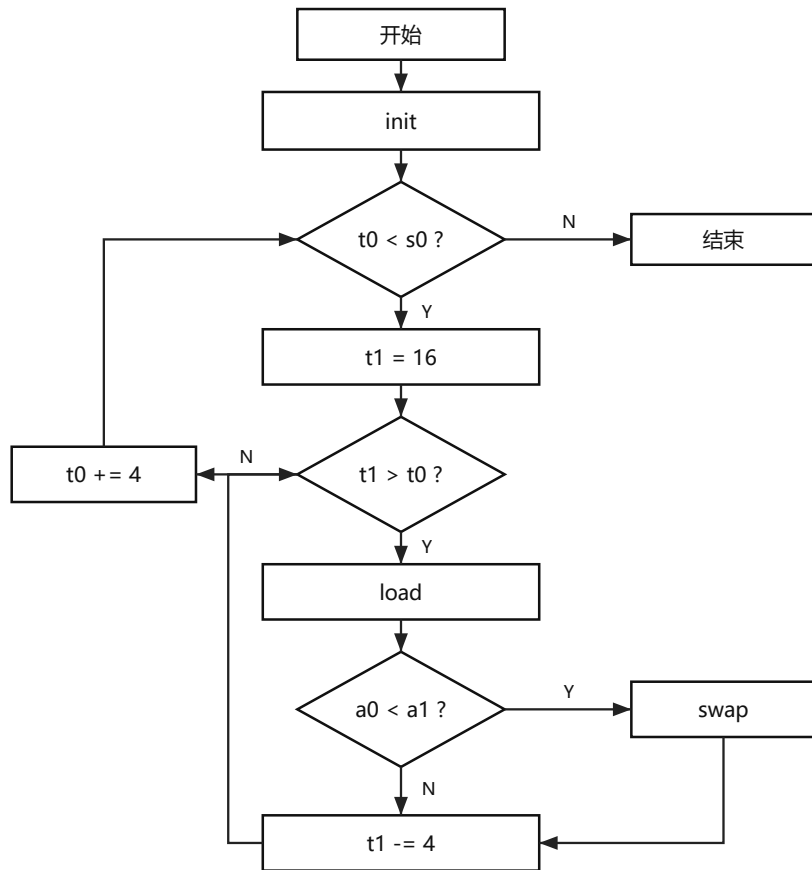


图 5-2 排序程序流程图

```

1 .data
2 list:
3     .word 4, 2, 5, 1, 3
4
5 .text
6 bsort:
7     xor  t2, t2, t2
8     addi s0, t2, 20
9     ori  t0, t2, 0
10    jal  end1
11 L1:
12    addi t1, t2, 16
13    jal  end2
14 L2:
15    lw   a0, 0(t1)
16    lw   a1, -4(t1)
17    bge  a0, a1, endif
18    sw   a1, 0(t1)
19    sw   a0, -4(t1)
20 endif:
21    addi t1, t1, -4
22 end2:
    
```

```

23    blt  t0, t1, L2
24    addi t0, t0, 4
25 end1:
26    blt  t0, s0, L1

```

5.2 机器码

指令存储器首地址处放置 00000000，在 CPU 中作为空指令跳过，后续 15 条指令对应排序程序主体。表 5-1 按指令字段给出机器码拆分结果。

表 5-1 排序程序指令与机器码

序号	指令格式	具体指令	funct7 (31-25)	rs2 (24-20)	rs1 (19-15)	funct3 (14-12)	Rd (11-7)	Op (6-0)
0	-	nop	0000000	00000	00000	000	00000	0000000
1	R	xor t2,t2,t2	0000000	00111	00111	100	00111	0110011
2	I	addi s0,t2,20	0000000	10100	00111	000	01000	0010011
3	I	ori t0,t2,0	0000000	00000	00111	110	00101	0010011
4	J	jal end1	0000001	01100	00000	000	00001	1101111
5	I	addi t1,t2,16	0000000	10000	00111	000	00110	0010011
6	J	jal end2	0000000	11100	00000	000	00001	1101111
7	I	lw a0,0(t1)	0000000	00000	00110	010	01010	0000011
8	I	lw a1,-4(t1)	1111111	11100	00110	010	01011	0000011
9	B	bge a0,a1,endif	0000000	01011	01010	101	01100	1100011
10	S	sw a1,0(t1)	0000000	01011	00110	010	00000	0100011
11	S	sw a0,-4(t1)	1111111	01010	00110	010	11100	0100011
12	I	addi t1,t1,-4	1111111	11100	00110	000	00110	0010011
13	B	blt t0,t1,L2	1111111	00110	00101	100	01001	1100011
14	I	addi t0,t0,4	0000000	00100	00101	000	00101	0010011
15	B	blt t0,s0,L1	1111110	01000	00101	100	11001	1100011

5.3 关键实现

立即数生成器按 imm_type 选择不同格式。B 型和 J 型立即数的最低位固定为 0，所以生成时直接把 1'b0 拼到最低位，再做符号扩展。

```

1 case (imm_type)
2     IMM_I: imm = {{20{instr[31]}}, instr[31:20]};
3     IMM_S: imm = {{20{instr[31]}}, instr[31:25], instr[11:7]};
4     IMM_B: imm = {{19{instr[31]}}, instr[31], instr[7],
5                 instr[30:25], instr[11:8], 1'b0};
6     IMM_U: imm = {instr[31:12], 12'b0};
7     IMM_J: imm = {{11{instr[31]}}, instr[31], instr[19:12],
8                 instr[20], instr[30:21], 1'b0};
9 endcase

```

分支单元只负责比较和选择下一条 PC。jal 被看作无条件跳转，同时写回选择器把 PC+4 写入 rd。这样分支判断和寄存器写回互不混在一起，顶层连线也更加清楚。

```

1 case (branch_op)
2     BR_BEQ: take_branch = (rs1_data == rs2_data);
3     BR_BNE: take_branch = (rs1_data != rs2_data);

```



```

4    BR_BLT: take_branch = (s_rs1 < s_rs2);
5    BR_BGE: take_branch = (s_rs1 >= s_rs2);
6    BR_JAL: take_branch = 1'b1;
7    default: take_branch = 1'b0;
8  endcase
9
10  assign next_pc = take_branch ? (pc + imm) : (pc + 32'd4);

```

第六章 测试

6.1 RARS 运行结果

在 RARS 中运行排序程序后，数据段前 5 个字变为 1, 2, 3, 4, 5，运行输出也显示排序后的数组，如图 6-3 所示。

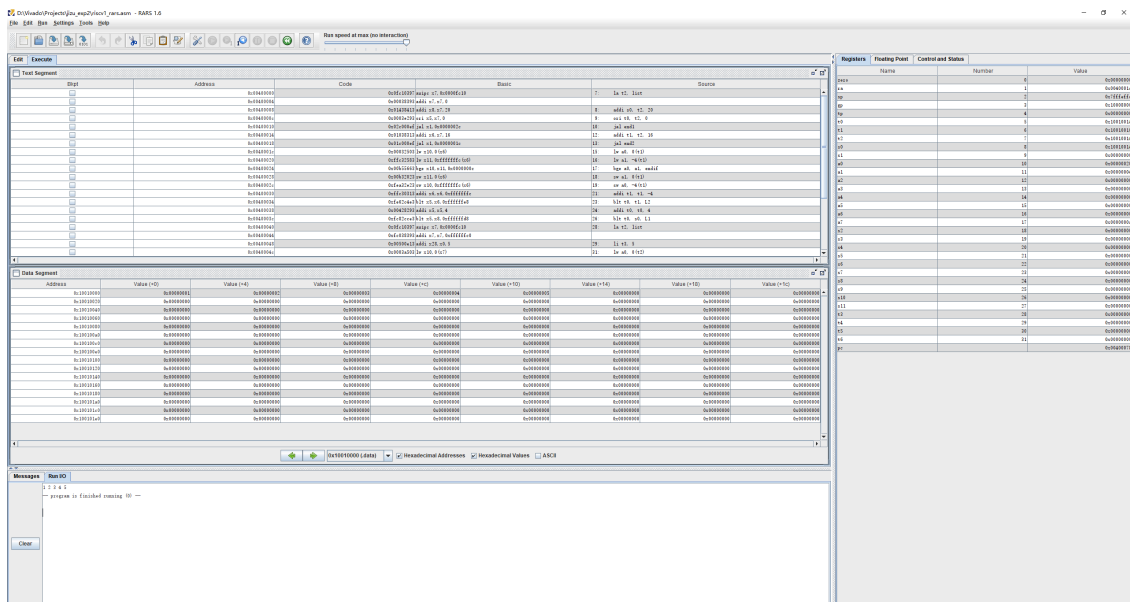


图 6-3 RARS 排序运行结果

这说明汇编程序本身完成了从小到大的排序。

6.2 CPU 仿真方法

Vivado 仿真中提供时钟和低有效复位，运行 1200 ns 后检查数据存储器前 5 个字。期望输出为：

```

1 sorted_data=1,2,3,4,5
2 SORT_PASS

```

Vivado 仿真波形见图 6-4。图中 clk 正常翻转，rst 在复位后保持为 1，pc 和 instr 随程序执行持续变化，mem_write 在交换数组元素时产生写脉冲。仿真后段 instr 和 alu_y 出现

x, 是因为排序程序已经结束而 PC 继续向后取到未初始化的空指令地址, 不影响前面已经完成的排序结果。最终 mem0 到 mem4 分别为 1, 2, 3, 4, 5。

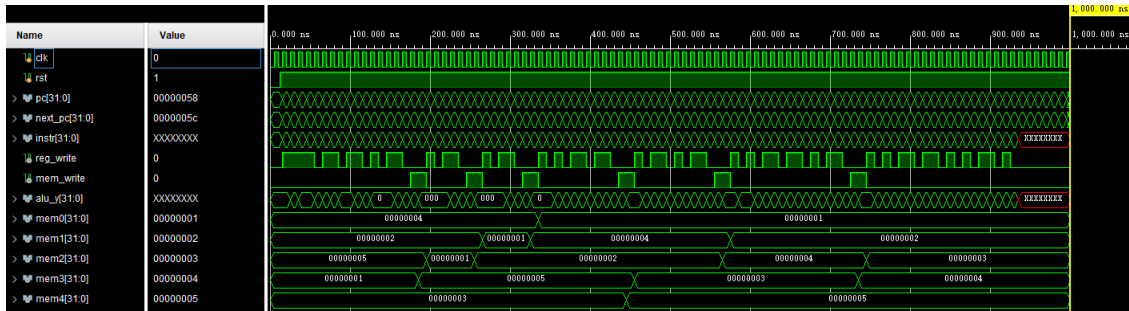


图 6-4 Vivado 排序仿真波形

控制台输出见 图 6-5, sorted_data=1,2,3,4,5 和 SORT_PASS 直接给出了自动检查结果。

```
run 1200ns
sorted_data=1,2,3,4,5
SORT_PASS
```

图 6-5 Vivado 控制台输出

6.3 结果对比

表 6-1 Vivado 与 RARS 结果对比

指令序列	单周期处理器仿真结果	RARS 仿真结果
xor; addi; ori	t2 清零, s0 得到数组边界 20, t0 初始化为 0	寄存器初值变化一致
jal end1	PC 跳转到外层循环条件判断位置	程序跳转到 end1 标签
addi t1,t2,16; jal end2	t1 指向最后一个数组元素, 随后进入内层条件判断	寄存器和跳转位置一致
lw a0,0(t1); lw a1,-4(t1)	从数据存储器读出相邻两个元素到 a0、a1	数据段对应元素读入寄存器
bge a0,a1,endif	若右侧元素不小于左侧元素, PC 直接跳过交换语句	分支判断结果一致
sw a1,0(t1); sw a0,-4(t1)	需要交换时, 两个相邻存储单元的值互换	数据段相同位置完成互换
addi t1,t1,-4; blt t0,t1,L2	内层指针左移, 满足条件时回到 L2	内层循环行为一致
addi t0,t0,4; blt t0,s0,L1	外层指针右移, 未到边界时进入下一轮	外层循环行为一致
程序结束后	mem0 到 mem4 为 1, 2, 3, 4, 5	数据段前 5 个字为 1, 2, 3, 4, 5

第七章 问题及解决方法

本实验中最容易出错的是分支立即数和访存地址。B 型立即数在指令中不是连续字段，需要重新拼接 `instr[31]`、`instr[7]`、`instr[30:25]` 和 `instr[11:8]`，如果把它当作普通 12 位立即数处理，循环跳转目标会偏移错误。访存部分也要注意当前数据存储器按 32 位字组织，因此地址低两位不用作数组下标，实际访问时使用 `addr[9:2]`。

另一个问题是复位和程序入口。测试平台使用低有效复位，PC 复位后从地址 0 开始取指。首条指令设为 `00000000`，CPU 将未知或空指令按默认路径处理为 `PC+4`，因此排序程序主体从下一条指令开始执行。

第八章 心得体会及总结

单周期 CPU 的结构看起来直接，但每条指令都要在同一个周期内走完对应的数据路径，因此控制信号的含义必须清楚。比如 `lw` 和 `sw` 都要用 ALU 做地址加法，但前者要写寄存器、后者要写数据存储器；分支指令不写寄存器，却会影响下一条 PC。把这些差异放进控制表之后，调试时就可以按“当前指令、控制信号、关键数据”顺序检查。

本次实现先保证排序程序涉及的指令能正确运行，再把同类指令补齐到控制器中。这样的顺序比较稳妥，因为排序程序同时覆盖循环、比较、访存和交换，能作为一个较完整的功能性测试。后续如果继续扩展 RV32I 指令，只需要在控制器、ALU 和立即数生成器中补充对应分支，并增加针对性测试即可。